

面向边缘智能的 AIOS 设计与优化

决赛进展汇报

Team Posad · 清华大学

RK3588 / OrangePi-5-Plus · 基线 rcore-os/tgoskits dev @73409e079 · 2026-08

目录

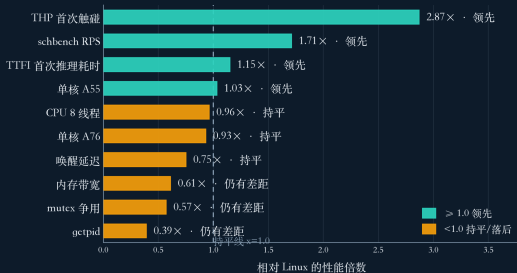
- ▶ 一 背景与目标 命题转变、机器人拾球回路、系统分层、基准与测量口径
- ▶ 二 系统能力与性能 观测、调度、系统调用、内存、频率、启动、显示、NPU、板级、应用
- ▶ 三 增量与展望 贡献矩阵、对比、分工、进度
- ▶ 四 方法与复现 AI 使用与方法论、主要贡献、后续工作、材料入口

顶部导航条标示当前所在段落。

主要工作与结果

跨子系统对等仪表盘

归一化性能比: 吞吐类 = Starry/Linux, 时延类 = Linux/Starry (数值越大越好)



注: 各项均为 RK3588 整机实测, 3 次重复取中位数

起点: 网球拾取机器人由 C 语言的 Linux 迁移至 Rust 实现的 StarryOS, 平台为 RK3588。

- ▶ **整机回路跑通** 追球到投放五阶段闭环在板上运行, TTFI 19.22→ 9.83s, 优于 Linux 11.28s
- ▶ **整 OS 向 Linux 对等** 两侧同一份 ELF, cpu 八线程 160→ 4987 ev/s, 为 Linux 5222 的 0.96 倍
- ▶ **多项已合并上游** 观测、调度、DVFS、驱动多项改动并入 rcore-os/tgskits

从初赛到决赛：整 OS 对等命题

初赛

- ▶ 重心在单条推理流水线
- ▶ 依靠应用侧取球循环与数据布局，将首帧延迟降至接近 Linux

决赛

- ▶ 命题转为整机对等，令 StarryOS 作为通用 OS
- ▶ 在调度、系统调用、内存、频率等与机器人无关的子系统直接对比 Linux
- ▶ 机器人为众多负载之一，考核对象为操作系统的通用性

两侧运行同一份遵循 Linux ABI 的可执行文件，避免以不可比的口径拼凑结论。

目标完成情况

目标域	结果 (StarryOS 优化前 → 后, 对照 Linux)	状态
观测 perf/ftrace	硬件 PMU perf 与 ftrace 上板, big.LITTLE 全量通过	已达
调度与大小核	cpu 八线程 160→4987 ev/s, 为 Linux 5222 的 0.96×	已达
系统调用入口	getpid 1200→431 ns, 为 Linux 167 ns 的 2.6× (由 7× 收窄)	已达
内存与分页	THP first-touch 0.8 到 1.3→0.030 s, Linux 0.086 s, 快 2.87× (领先)	已达
频率电压	CPU 调频调压经 PVTPLL 环长对齐 Linux (逐核持平); 内存写 15040→34330 MiB/s (0.61×, DDR 变频属固件外因)	已达
启动 TTFI	19.22→9.83 s, 优于 Linux 11.28 s (领先)	已达
显示栈	VOP2 + HDMI-QP + HDPTX 上板点亮, ROPLL 锁定验证通过	已达
QEMU NPU 模型	RKNPU 功能级仿真合入, 106 个 qtest	已达
机器人应用	网球拾取机器人五阶段闭环在 RK3588 真机完整运行, 感知到运动控制全链路	已达

系统分层总览与术语



- ▶ StarryOS 复用 ArceOS 模块层，补齐 Linux 兼容的系统调用、进程与信号，按设备类型分层驱动
- ▶ 本队改动集中于调度、DVFS、分页、观测四层，另含若干 RK3588 驱动

术语一次讲清：

big.LITTLE = 4×A55 小核 (cpu0-3) + 4×A76 大核 (cpu4-7)；

DVFS = 动态调频调压；THP = 透明大页；

TTFI = 上电到首帧推理。

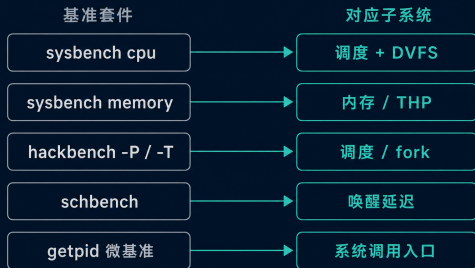
网球机器人与拾球回路



- ▶ 闭环起于相机成帧，依次经 JPU 解码、RGA 预处理、NPU 推理，直至定位与运动控制，再返回相机
- ▶ 帧到指令延迟即闭环运行一周的端到端时间
- ▶ 追球交由 NPU 上的单类 YOLOv8，寻桶采用整型 HSV 红色掩膜
- ▶ 感知结果送入纯逻辑状态机，生成底盘与舵机指令

基准套件与测量口径

基准套件与子系统对照



每项基准均在同一开发板上对同一 Linux 基线独立复测

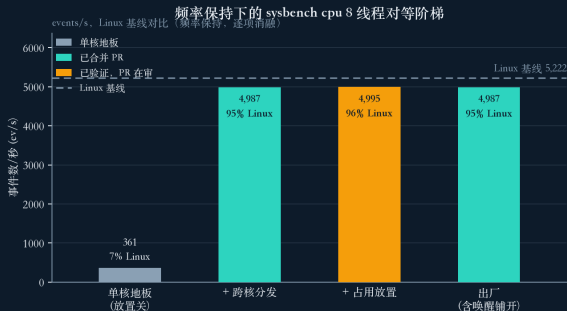
- ▶ 每个子系统采用各自业界基准，各配一条同机同频 Linux 基线，两侧运行同一份 ELF
- ▶ cpu 考察调度与 DVFS，memory 考察分页与 THP，hackbench 考察 fork 与并发，schbench 考察唤醒延迟，getpid 考察系统调用入口
- ▶ 每点取 $N \geq 5$ 的 p50，两次读数漂移超 $\pm 2\%$ 判为噪声重测
- ▶ 主线用累计阶梯逐层叠加，相互影响者再单独正交消融

观测能力：硬件 PMU perf 与 ftrace

- ▶ 后续数据均来自测量，测量依赖工具；决赛前 StarryOS 尚无基于硬件 PMU 的 perf，热点难以定位到层
- ▶ 构建一整套硬件 PMU perf：单核 PMUv3、big.LITTLE 分簇、软件与 cache 事件、事件组、调用栈、kprobe/tracepoint、ftrace
- ▶ 未经修改的上游 perf 6.1.0 二进制直接产出 perf.data、report 与 top；板级 big.LITTLE 矩阵 39/0、特性矩阵 56/0
- ▶ 受硬件限制的 PEBS 与 BRBE 类特性，按 Linux 在同一 SoC 上的行为如实拒绝，未伪造读数

ftrace 全量函数跟踪在真实 8 核开发板活锁（QEMU 因 smp1 未暴露），过滤式函数跟踪为板上与 QEMU 均实用的模式。

调度：sysbench cpu 对等阶梯



注：单独启用 wake_affine 就近唤醒会因并发线程聚拢而将 8 线程吞吐降至 3523，出厂配置的唤醒铺开将其恢复至 4987。

- ▶ 决赛初整机压在单个 A55，八线程仅 160 ev/s，落后 Linux 约三十倍（下图为频率固定下的隔离阶梯，地板 361）
- ▶ 固定顶频以隔离放置层：跨核分发（#1656）把八线程由单核地板 361 抬到 4987 ev/s，为最关键一层
- ▶ 叠加占用感知放置续进至 4995 ev/s，增益已属小幅；出厂配置八线程 4987 ev/s，为 Linux 5222 的 0.96 倍，判定持平

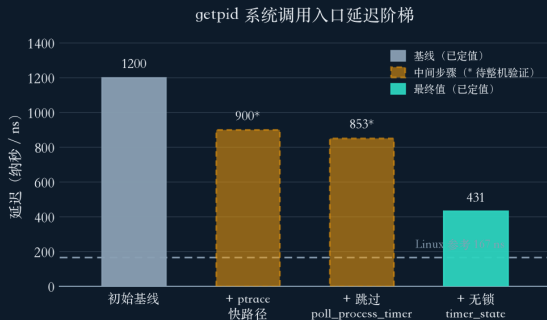
调度：核驻留与唤醒延迟

核心占用率：负载均衡优化前后对比



- ▶ 八线程曾塌陷至 3 个 A76、A55 全程空闲，根因为唤醒聚簇
- ▶ 对齐 Linux
`select_idle_sibling`: 仅 `occ==0` 时保留 `last_cpu`，否则选最空闲核，驻留随即均匀
- ▶ `wake_affine` 将 `m1t4` 唤醒 `p50` 由 1042 降至 8 μ s，接近 Linux 6 μ s，判定持平
- ▶ 出厂启用 `idle-poll` 后，并发工作线程经 `select_idle_sibling` 铺开，`schbench` RPS 339 反超 Linux 199 (1.71 $\times$)

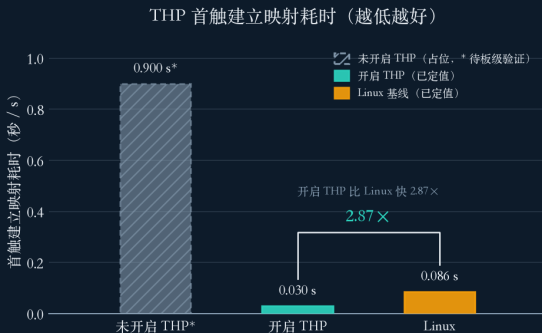
系统调用入口：getpid 微阶梯



* 初始基线与中间两级为实现当期单次测量，最终值与 Linux 参考取三次重复中位数

- ▶ 空系统调用 getpid 基线 1200 ns，Linux 167 ns，相差约 7 倍，开销集中于进入与返回路径
- ▶ 三层剥离每次调用均付而多数不用的固定成本：ptrace 快路径 -25%、跳过 poll_process_timer -17%、无锁 timer_state 粗 tick 记账 -50%
- ▶ getpid 降至 431 ns，相对 Linux 由 7 倍收窄至 2.6 倍，stime 秒占比 0.71 对齐 Linux 0.72
- ▶ 残余 431 ns 为 uctx.run 陷入的架构地板，三套系统共用，无冗余可摘

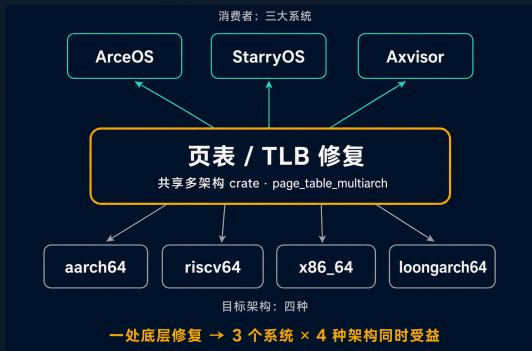
内存：THP first-touch (领先)



* 未开启 THP 为占位值，等待整机 oblation 复测确认 (不影响开启 THP / Linux 两侧已定值)

- ▶ 首触路径慢：128 MB fault-in 需 0.8 到 1.3s，Linux 0.086s，逐页约 9 到 15 倍
- ▶ 三级压缩：新建映射跳过广播 TLB flush、aarch64 dc zva 块清零、THP-lite 2 MB 私有匿名提升
- ▶ 2 MB 私有匿名首触降至 0.030s，Linux 0.086s，快 2.87 倍 (领先)
- ▶ 收益来自硬件层面 (更少 TLB 广播、块清零、每 2 MB 少 512 次缺页)，Linux 侧缺乏等价用户态手段

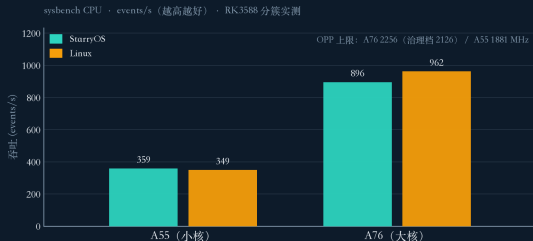
正确性：页表与 TLB 修复的波及面



- ▶ 提升大页、跳过失效、引入 2MB 块时，多代理对抗审查（每项 7 到 26 人）发现一批潜伏的 SMP 与 UAF 缺陷
- ▶ 主要一处：not-present 巨页块被误当页表遍历，unmap 时重复释放与 COW 共享帧构成 UAF；以 `GenericPTE::is_table()` 统一判定修复
- ▶ aarch64 侧修复 PTE 写序缺 `dsb ishst`、本地 `vmalle1` 应为广播、拆分对 OOM 不原子（波及面见左图）

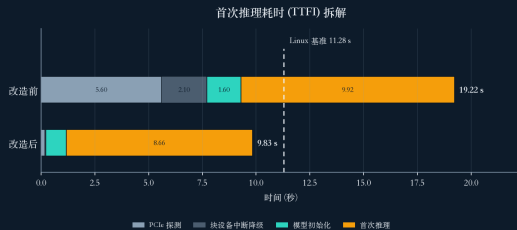
频率电压：DVFS / PVTPLL 与 DDR 变频固件外因

DVFS 分核性能追平 Linux



- ▶ RK3588 CPU 时钟为电压耦合的 PVTPLL，频率随轨压变化，CPU CRU 由 BL31 保护
- ▶ #1657: 按簇 ondemand governor + PMIC 电压标定 (A76 走 I2C、A55 走 SPI)，事务式先电压后时钟
- ▶ 环长调节: 更高 SCMI 速率将环由 17 重编为 11，同压更高频; A76 顶 2126 MHz、A55 顶 1881 MHz，八线程达 $0.96 \times$ Linux
- ▶ **DDR 变频**走 Rockchip SIP DRAM 接口，主线 TF-A 缺该 EL3 处理器，驱动侧已就位，停在探测态，受主线固件外因所限

启动：首帧推理 TTFI (领先)



各阶段均分为微基准。待整板复测确认更新 (改造前/改造后/Linux 总耗时已由串口时间戳观测确认)

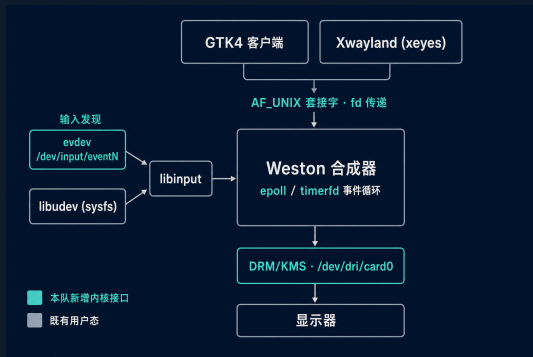
- ▶ 口径：内核自 U-Boot 接管起至 live 首帧 RKNN 推理完成
- ▶ 提交版本 19.22s 慢 Linux 约 1.7 倍；经根因定位与约十一处公平改动降至 9.83s，优于 Linux 11.28s (领先)
- ▶ 内核侧两段：PCIe 空槽串行探测约 5.6s (次核前置 + FDT 借用 + 空槽快返)，块 IO 完成中断回退约 2.1s
- ▶ 应用侧最大一段：相机边流边初始化使 rknn_init 饥饿，先加载模型再启相机将 model_init 由 16870 降至 140ms

原生显示栈 VOP2 / HDMI-QP / HDPTX (上板点亮验证)

Crate	职责	主机测试
rockchip-vop2	VP 时序、Esmart0 窗口、VP0→HDMI0 modeset	23
dw-hdmi-qp	工作模式、AVI infoframe	6
rockchip-hdptx-phy	193 条 PHY 写、ROPLL 像素时钟	7
rockchip-soc CRU	显示时钟树扩展	3

- ▶ 从零实现 `#![no_std]` RK3588 显示栈，Qt6 时钟经 `/dev/fb0` 上屏；四 crate 合计 39 项主机测试通过
- ▶ **上板点亮验证通过** (RK3588)：HDPTX ROPLL 锁定 (GRF STATUS=0x0e)，VP0 1080p60 光栅实时扫描，`/dev/fb0` 绘出彩条，Qt6 时钟原生运行
- ▶ HDPTX 193 条寄存器写逐字移植主线，判据为逐位一致与两次 GRF 锁定轮询
- ▶ 上板前对抗复核已修两个 HIGH (FB 缓存一致性、config-done 锁存)

图形合成器：Weston 于 StarryOS（已合并）

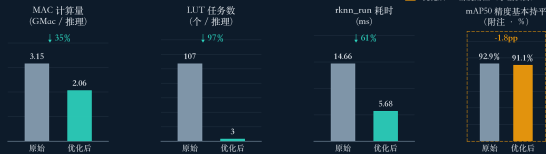


- ▶ 在 StarryOS 上跑通 Weston：客户端经 AF_UNIX 提交窗口缓冲，DRM/KMS 扫描输出，epoll/timerfd 事件循环
- ▶ 逐层补齐内核接口：/dev/dri/card0 原子 KMS 与 GEM dumb、多 eventN 输入、AF_NETLINK 网络配置、AF_UNIX 传 fd、epoll 水平触发改如实就绪
- ▶ 四架构验证：GTK4 客户端上屏经 VNC 远看、WAYLAND_TEST_PASSED 通过、Xwayland 跑通 xeyes

QEMU RKNPU 设备模型与模型协同优化

模拟器实测：模型结构协同优化前后对比

QEMU RKNPU 模拟器实测 (board-free mode) · INT8 · 285 图 / 382 框保固集 · conf 0.25 / NMS IoU 0.45



影响最大的一步: SiLU → ReLU (消除大量 LUT 查表算子, rknn_run 近乎减半)

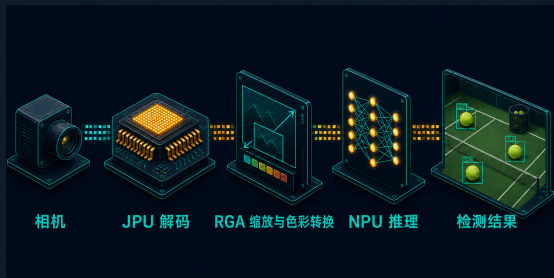
数据来自 QEMU RKNPU 功能模拟数据 (profiling, 模拟层已跑 process/mem/proc/19 / > 介入上层, profiling 数据有一个库层的合并请求提供, 均为模拟器内部数据, 非真机的融合位 "self-fuse")

- ▶ 本队将闭源 RK3588 RKNPU 实现为功能级 QEMU 模型 (#19、#26, 106 qtests), 真实执行 INT8/FP 并接入 IOMMU
- ▶ 模型将板上调试的加速器转为 board-free、寄存器精确的开发、CI 与优化载体
- ▶ 逐层重塑网球检测模型: SiLU 改 ReLU 使 LUT 任务 107→3、rknn_run 减半, 叠加输入缩放、类别头收窄、头框合并 (左图为模拟器实测, 非真机)

板级 I/O 与外设驱动（已合并上游）

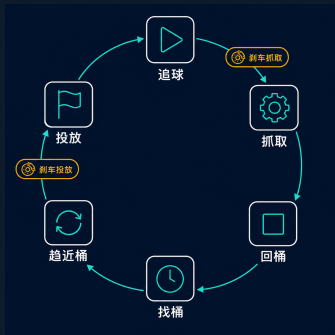
- ▶ 每项上板数据以 RK3588 能被稳定驱动为前提；该层由本队仿真与板级侧完成并合入上游，不含对 Linux 的性能主张
- ▶ 感知与执行通路：UART (#1921)、USB 串口 tty (#1378)、TTY 刷新唤醒 (#1922)、PWM sysfs (#1468)、reboot(2) (#1358)、UVC 生命周期 (#1924)、UVC 迁移 ax-sync (#1961)
- ▶ 从网卡上电到板上 SSH：pinctrl IOMUX 偏移修正 + regulator GPIO 上电 + RTL8125 枚举 + rtnetlink IPv4 (#1497)，SSH 成为第二条登录通路
- ▶ 暖重启回路全程不碰电源键，网络 scp 加 fatload 将 14 MB FIT 落盘由 90s 以上压到约 1 到 1.4s

视觉推理链路：从落后 6.3 倍到对等

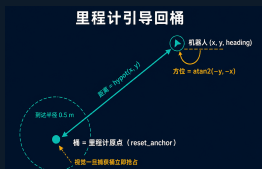


- ▶ 同一份 ELF 两侧对测，StarryOS 起点端到端落后 Linux 约 6.3 倍，热点散在链路各阶段
- ▶ 绑定 A76 大核：640×640 端到端 162.7→55.2 ms，三大 CPU 阶段各降约 3 到 5 倍
- ▶ 整数取回路径（want_float=0）使 outputs_get 约 12→1 ms，285 张测图逐张一致
- ▶ 原生 ReLU 模型将 480×640 p50 30.6→11.75 ms 追平 Linux，再加 RGA 零拷贝逐帧命令降到约 9.65 ms

机器人拾球应用：状态机与里程计回桶



- ▶ 应用为一份 C++ 状态机，主机 dry-run 与板端同源，仅替换电机与机械臂后端
- ▶ 六状态拾球闭环（追球、抓取、回桶、寻桶、趋近、投放）纯逻辑非阻塞，定时时相由单调时钟推进而与相机 FPS 解耦
- ▶ 里程计引导回桶：投放时 `reset_anchor` 把桶设为原点，航位推算按 `distance` 与 `bearing` 回锚点，视觉一旦捕获桶即抢占
- ▶ 慢速轮速遥测移出控制回路，由后台线程 100 ms 周期采样并更新位姿快照，控制回路每轮只取一次快照



经验证暂不适用当前平台的尝试

尝试	现象	为何无收益或回退
全量 work-stealing 均衡器	单线程 350→161 (0.46×)，饱和无收益	七个空闲核反复迁移唯一工作线程，无核心保持缓存热度
自适应 haltpoll	未稳定优于固定 50 μs	停机时长信号受 SGI-WFI 交互干扰而失真
early-MMU 启动重排	4.446 vs 4.46 s	低频启动窗口受 CPU 制约，缓存无从节省
第三 NPU 核	由两核到三核无收益	该模型约占一个核，增配核心无法占满

多数尝试在更换硬件类型或补足前置条件后仍有望转为正收益，适用边界已随各条列出。

按轴诚实图例：StarryOS 优化前 → 后，对照 Linux

轴（单位）	优化前	优化后	Linux	判定
THP first-touch (s)	0.8 到 1.3	0.030	0.086	领先 2.87×
启动 TTFI (s)	19.22	9.83	11.28	领先
A55 单核 (ev/s)	约 251	359	349	领先 1.03×
schbench RPS	231	339	199	反超 1.71×
A76 单核 (ev/s)	约 693	896	962	持平 0.93×
cpu 八线程 (ev/s)	160	4987	5222	持平 0.96×
唤醒 p50 (μs)	1042	8	6	持平
getpid (ns)	1200	431	167	收窄 2.6×
mutex 8T (s)	1.69	0.85	0.481	收窄 1.76×
内存写 (MiB/s)	15040	34330	56046	收窄 0.61×

“收窄”指相对 Linux 的差距已大幅缩小（getpid 由 7× 收至 2.6×，mutex 由 3.5× 收至 1.76×）；内存写受 DDR 变频固件外因所限。

基础版本与增量贡献

PR 归属矩阵：子系统 × 贡献者

子系统	系统内核侧	仿真与板级侧
perf	■ #1395 ○ #1577 #1601 #1602 #1603	-
调度	■ #1495 ○ #1656 #1658	-
DVFS	■ #1657	-
内存	- 会话中（未开 PR）	-
正确性	- 会话中（未开 PR）	-
驱动 RGA/JPU	■ #1388 #1456	-
板级 I/O	-	■ #1358 #1378 #1921 #1922 #1468 #1924 #1961 #1497 #1963
QEMU-NPU	-	■ processmission/qemu#19 #26
机器人	-	- 会话中（未开 PR）

- ▶ 基线冻结于 rcore-os/tgoskits 的 dev 分支 commit 73409e079，每处原子改动以此为起点派生为一个 PR
- ▶ 拆分至可独立回归的粒度，使每条改动可对照固定基线核对得失，评审仅需关注单一逻辑
- ▶ 已合并覆盖观测、调度、DVFS、驱动与外设初始化；在审含多核 perf 四件与调度分布（#1656，把八线程由平线 361 抬到 4987 ev/s）
- ▶ 第三梯队三项（THP 首触、页表正确性、机器人应用）已在集成分支实现并经板级验证，待拆分为独立 PR

与类似工作的对比

- ▶ 多数 Rust OS 工作在 QEMU 或综合基准上自证，少有在真实边缘 SoC 上与同机 Linux 运行同一份 ABI 可执行文件的逐轴对照
- ▶ 本工作在 RK3588 真机上以同一份 ELF、同机同频、 $N \geq 5$ 的 p50 对照 Linux，覆盖调度、系统调用、内存、频率、启动五个轴
- ▶ 加速器侧将闭源 RKNPU 实现为功能级 QEMU 模型（106 qtests），使 NPU 驱动与模型优化脱离板子，属可复用产物
- ▶ 定调诚实：给出的是对等与少数领先，并如实标注外因（DDR 固件）与后续优化方向（系统调用入口、锁竞争）

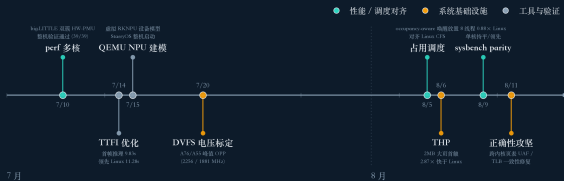
队员分工 (Team Posad, 三名成员)

成员	负责模块与交付物
洪世金 (队长)	perf 观测 (#1395/#1577)、调度器与大小核 (#1495/#1656)、DVFS 与 cpufreq (#1657)、内存与 THP、页表与 TLB 正确性、RGA2 (#1388) 与 JPU (#1456) 驱动、原生显示栈 VOP2 / HDMI-QP / HDPTX、sysbench (#1658)、报告撰写与整体协调
王乙凡	QEMU RKNPU 功能级模拟器：全流水线 + IOMMU + INT8/FP 执行 (#19, 106 qtests)，多核协同 (#26)；机器人拾球应用 (与徐子航共同)
徐子航	板级 I/O 与网络：UART (#1921)、USB tty (#1378)、PWM (#1468)、rtnetlink 与 SSH (#1497)；UVC 相机 (#1924/#1961)、reboot (#1358)、GEM cache mmap (#1364)；机器人拾球应用 (与王乙凡共同)

开发进度与完成情况

开发时间线（7月 - 8月）

StarryOS / RK3588 关键里程碑。日期为该能力落地的代表性节点



里程碑日期取自开发过程中该项工作首次整机/QEMU 验证通过的节点；具体数据随复现可能微调（当前均为已定值，无 ablation 依赖）。

- ▶ 决赛约一个月（07-01 至 08 月中），三名队员协作，各项以真板验证为准；左图为七周里程碑，自 perf 多核起至页表正确性与原子 PR
- ▶ 落地并验证：PMU perf、QEMU RKNPU、SMP 分布、DVFS 环长、占用调度、THP 首触、CLONE_VM 快路径、系统调用下降、TTFI 领先
- ▶ 三项如实回退或受外因所限：工作窃取均衡器抖动回退、early-MMU 无收益回退、DDR 变频受主线固件外因所限

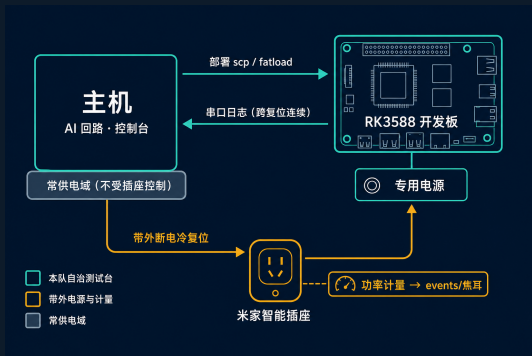
AI 使用说明

- ▶ 本月工作量超出同等时间手写代码所能覆盖的规模，方法为人掌架构、AI 执行闭环的分工
- ▶ 人一侧确定方向、划定原子改动的边界、制定验收标准；AI 一侧在真机与 QEMU 上迭代实现，运行至判定门
- ▶ 全部验收标准落实为机器可判定的判定门，一次运行输出一个可对比数值，取代肉眼读取日志得出结论
- ▶ 边界清晰：架构决策与最终验收由人负责；三名队员能在无 AI 在场时口头阐明每处关键设计，并当场修改对应内核代码

AI 方法论与正确性保障

- ▶ 四类机器可判定判定门：sysbench 对等回归门、schbench 延迟阈值门、板上无死锁的多智能体评审门、s_frac 计时正确性 oracle
- ▶ 触及 SMP 同步的改动先拆为单一关注点，再交 7 到 26 名独立评审 agent 对抗复核
- ▶ 实例：cpufreq apply_opp 静默吞掉电压写失败（高频欠压硬件风险），26 人评审查出，改 #[must_use] fail-closed
- ▶ 测量否掉设想：DDR 带宽差经板测否定 THP/tick 假设，真因是 DDR 固件；fork EFAULT 定位到 COW u8 溢出

真机自治测试台：带外电源控制



- ▶ 暖重启只在被测系统仍响应软件命令时才恢复，但这条闭环恰用于验证会挂死内核的改动
- ▶ 恢复原语须独立于被测软件状态，唯一如此者是电源：计量式智能插座（米家智能插座）经 miIO 本地协议带外冷复位，不经云端
- ▶ 分级恢复：L0 暖重启 → L1 插座硬断电冷启 → L2 U-Boot 金身回退 → L3 人工
- ▶ 冷启动路径的上板点亮会话即在该测试台完成，插座硬断电保证挂死可无人值守恢复

主要贡献与创新（呼应主要工作与结果）

- ▶ **观测从无到有** StarryOS 首套硬件 PMU perf，直接运行上游 perf 二进制，big.LITTLE 56/0
- ▶ **整 OS 对等** 跨核分发 + 占用放置 + PVTPLL 环长调节，cpu 八线程 160→ 4987 ev/s ($0.96\times$ Linux)，单核持平至小幅领先
- ▶ **少数领先** THP 首触快 Linux $2.87\times$ ，TTFI 9.83s 优于 11.28s
- ▶ **方法与产物** 闭源 RKNPU 的功能级 QEMU 模型 (106 qtests)，以及人掌架构、AI 闭环、机器可判定判定门的开发范式

后续工作与可复现

- ▶ **后续方向** 调度全量均衡（迁移代价门限与迟滞）、DVFS Phase 2 电压闭环加温控、DDR SIP 换 rkbins BL31、THP 通用化（首触扩展为通用大页，补齐后台合并与 madvise）、NPU 三核上真机并上游化 QEMU 模型
- ▶ **诚实口径** 全部性能数为 RK3588 整机三次重复中位数，对照同机同频 Linux；两侧运行同一份遵循 Linux ABI 的可执行文件
- ▶ **可复现** 数字集中于 data/numbers.tex 宏，图经 render.py 由 JSON 溯源，scripts/board/boardctl.py 暖重启回路，lint.sh 校验字号、口径与调色板一致性

材料入口与致谢

- ▶ **代码** rcore-os/tgoskits (已合并 PR)、processmission/qemu (#19、#26 RKNPU 模型)、团队 fork (在审 PR)
- ▶ **报告与幻灯** 决赛技术报告与进展汇报见 docs/final-round/, 全部数字与图表经 numbers.tex 单一数据源溯源
- ▶ **板级复现** scripts/board/boardctl.py 暖重启回路, perf 与 sysbench 运行上游同名二进制
- ▶ **致谢** 感谢 rcore-os 社区评审与竞赛组委会